

SIMULADO 1: ESTRUTURAS & ALGORITMOS

5 Questões · Nível Intermediário/Avançado · Duração recomendada: 2h 30min

REGRAS OFICIAIS E INSTRUÇÕES DO SIMULADO

- Condições de Prova (Sem Consulta):** Resolva todas as questões sem consultar materiais externos, internet, documentação ou Inteligências Artificiais. O simulado simula a prova presencial do Desafio PUC-Rio.
- Formato das Entradas e Saídas:** Use exclusivamente ``input()``, ``int(input())`` e ``float(input())``. Não inclua mensagens nos prompts (ex.: use ``input()``, NUNCA ``input('Digite algo: ')``). A saída deve corresponder EXATAMENTE ao formato especificado.
- Testes Obrigatórios Antes do Envio:** Antes de enviar seu código final, você deve testar o programa com os 4 exemplos fornecidos na tabela E criar **2 casos de teste adicionais por conta própria** (sendo pelo menos 1 caso de borda/limite).
- Metodologia de Correção e Escada de Ajuda:** Envie o código de cada questão em Python 3 assim que concluir. Caso haja erros de lógica ou execução, utilizaremos a *Escada de Ajuda* com pistas progressivas para você descobrir e corrigir a falha.
- Gestão do Tempo e Raciocínio:** Antes de escrever o código, leia o enunciado atentamente e responda mentalmente às 3 perguntas da seção '*Antes de programar, pense*'. O tempo ideal de resolução para este simulado de 5 questões é de 2h 30min.

■ ESTRATÉGIA PARA O NÍVEL INTERMEDIÁRIO / AVANÇADO

- Atente-se a detalhes como maiúsculas/minúsculas, espaços e remoção de caracteres.
- Verifique com cuidado a ordenação de filas e matrizes (índices 0 e N-1).
- Crie testes para casos onde a entrada possui valores zerados, vazios ou no limite superior.
- Lembre-se: no Desafio PUC-Rio, a clareza do raciocínio e a exatidão da saída valem a pontuação máxima!

Questão 1 — Frequência e Substituição em Strings

Um sistema de análise textual precisa identificar a palavra mais frequente em uma frase e gerar uma versão censurada da frase. A contagem de frequência deve **desconsiderar a diferença entre maiúsculas e minúsculas** (ex.: 'Python' e 'python' são a mesma palavra). Se houver empate na maior frequência, deve-se escolher a palavra que **aparece primeiro** na frase. Em seguida, todas as ocorrências dessa palavra na frase devem ser substituídas pela tag [MAIS_FREQUENTE], preservando as demais palavras e seus formatos originais.

Entrada: Uma única linha contendo uma string S com palavras separadas por espaço (sem pontuações complexas).

Saída: Duas linhas:

- Linha 1: Palavra: <palavra_em_minusculo> | Ocorrencias: <qtd>
- Linha 2: A string S modificada com a tag [MAIS_FREQUENTE] nas posições correspondentes.

#	Entrada (S)	Saída Esperada
1	o gato o cao o gato	Palavra: o Ocorrencias: 3 [MAIS_FREQUENTE] gato [MAIS_FREQUENTE] cao [MAIS_FREQUENTE] gato
2	Python eh Python e Python eh bom	Palavra: python Ocorrencias: 3 [MAIS_FREQUENTE] eh [MAIS_FREQUENTE] e [MAIS_FREQUENTE] eh bom
3	A B C A B	Palavra: a Ocorrencias: 2 [MAIS_FREQUENTE] B C [MAIS_FREQUENTE] B
4	puc rio puc rio	Palavra: puc Ocorrencias: 2 [MAIS_FREQUENTE] rio [MAIS_FREQUENTE] rio

Antes de programar, pense:

1. Como converter a frase para minúsculas para contar frequências sem perder as maiúsculas da frase original na substituição?
2. Como desempatar de forma simples caso duas palavras tenham a mesma frequência máxima?
3. Como garantir que a substituição ocorra em palavra completa e não em subpalavras?

Questão 2 — Matriz e Diagonal Dominante

Em matemática computacional, uma matriz quadrada $N \times N$ é dita **estritamente diagonal dominante** se, para cada linha i , o valor absoluto do elemento na diagonal principal ($|A[i][i]|$) for **estritamente maior** do que a soma dos valores absolutos de todos os outros elementos daquela mesma linha.

Escreva um programa que leia a dimensão N e os elementos da matriz, verifique se ela cumpre a propriedade e informe o maior elemento encontrado na diagonal principal.

Entrada: A primeira linha contém um inteiro N ($N \geq 2$). Seguem N linhas, cada uma com N inteiros separados por espaço.

Saída: Primeira linha: SIM se a matriz for estritamente diagonal dominante ou NAO caso contrário. Segunda linha: O

maior valor presente na diagonal principal.

#	Entrada	Saída Esperada
1	3 5 1 1 2 6 3 1 2 4	SIM 6
2	2 3 3 1 4	NAO 4
3	3 10 2 3 -1 8 4 0 5 -9	SIM 10
4	2 5 10 1 2	NAO 5

Antes de programar, pense:

1. Qual função em Python calcula o valor absoluto de um número? (Dica: `abs()`).
2. Como somar os elementos de uma linha excluindo apenas a posição da diagonal principal ($i == j$)?
3. Se apenas uma linha falhar no teste de estritamente maior, a matriz inteira é considerada não dominante?

Questão 3 — Fila de Atendimento com Prioridade Proporcional

Um centro de atendimento organiza seus clientes em duas filas: **Prioritários** (P, idade ≥ 60) e **Normais** (N, idade < 60). A regra de chamada é: para cada **2 clientes prioritários** atendidos, atende-se **1 cliente normal**. Caso não haja prioritários suficientes na fila, atendem-se os normais disponíveis; se não houver normais, atendem-se os prioritários restantes.

Entrada: A primeira linha contém um inteiro K (quantidade de clientes). Seguem K linhas, cada uma com o formato `tipo nome idade` (onde `tipo` é 'P' ou 'N', `nome` é uma string sem espaços e `idade` é um inteiro).

Saída: K linhas contendo os nomes dos clientes na ordem exata em que serão chamados para atendimento.

#	Entrada	Saída Esperada
1	4 P Ana 65 N Bob 25 P Caio 70 N Bia 30	Ana Caio Bob Bia

#	Entrada	Saída Esperada
2	5 N Joao 20 P Maria 80 P Jose 75 P Ana 62 N Leo 30	Maria Jose Joao Ana Leo
3	3 N Bia 20 N Caio 22 N Dani 25	Bia Caio Dani
4	3 P Ana 70 P Bob 80 P Caio 65	Ana Bob Caio

Antes de programar, pense:

1. É melhor separar a entrada em duas listas (uma de prioritários e uma de normais) logo na leitura?
2. Como controlar a proporção 2 para 1 em um laço while enquanto houver clientes em qualquer uma das filas?
3. O que acontece quando a fila de prioritários acaba mas ainda existem 3 clientes normais?

Questão 4 — Intersecção e Compressão de Intervalos

Dadas duas listas de inteiros A e B , determine a intersecção entre elas (elementos presentes em ambas as listas), eliminando duplicatas e ordenando em ordem crescente. Em seguida, gere uma versão comprimida da sequência resultante agrupando números consecutivos no formato inicio-fim.

Exemplo de compressão: A sequência 2, 3, 4, 8 torna-se 2-4, 8.

Entrada: Duas linhas. A primeira contém o tamanho N seguido dos N inteiros da lista A . A segunda contém o tamanho M seguido dos M inteiros da lista B .

Saída:

- Linha 1: Elementos da intersecção ordenados e sem duplicatas separados por espaço (ou VAZIO se for vazia).
- Linha 2: Sequência comprimida com vírgula e espaço entre grupos (ou NENHUM se for vazia).

#	Entrada	Saída Esperada
1	5 1 2 3 5 6 6 2 3 4 5 6 7	2 3 5 6 2-3, 5-6
2	4 10 20 30 40 3 5 15 25	VAZIO NENHUM
3	6 1 2 3 4 7 8 5 2 3 4 8 9	2 3 4 8 2-4, 8

#	Entrada	Saída Esperada
4	5 5 1 2 3 4 5 1 3 5 7 9	1 3 5 1, 3, 5

Antes de programar, pense:

1. Como remover duplicatas e ordenar uma lista em Python? (Dica: `sorted(list(set(interseccao)))`).
2. Como identificar quando um bloco de números consecutivos começa e termina ao percorrer a lista tratada?
3. Como tratar elementos isolados (sem consecutivos vizinhos) para que fiquem sem hífen na saída?

Questão 5 — Compressão RLE e Validação de Checksum

O algoritmo *Run-Length Encoding (RLE)* comprime sequências de caracteres consecutivos idênticos substituindo a sequência pelo caractere seguido da quantidade de repetições (ex.: 'AAA' torna-se 'A3').

Além de comprimir a string *S* (formada por letras maiúsculas), o programa deve calcular um **Checksum de Validação** somando o produto da quantidade de cada bloco pelo valor alfabético do caractere (onde 'A'=1, 'B'=2, ..., 'Z'=26).

Entrada: Uma linha contendo a string *S*.

Saída:

- Linha 1: String comprimida no formato RLE.
- Linha 2: O valor inteiro do Checksum calculado.

#	Entrada (S)	Saída Esperada
1	AAABBC	A3B2C1 10
2	AAAAA	A5 5
3	XYZ	X1Y1Z1 75
4	AABBCC	A2B2C2 12

Antes de programar, pense:

1. Como converter uma letra maiúscula para sua posição no alfabeto (1 a 26) usando `ord(c) - ord('A') + 1`?
2. Como acumular o caractere atual e a contagem ao percorrer a string caractere a caractere?
3. Não se esqueça de processar e imprimir o último bloco de caracteres após o término do laço principal!